

COMP229 – Web Application Development

Assignment 2 - Portfolio Backend CRUD with MongoDB

Due Week #5

Maximum Marks: 100

Overview: Develop the Node.js Express REST APIs that interact with a MongoDB database using the Mongoose ODM for your Portfolio application. The frontend created in Assignment 1 will be integrated with this backend in subsequent assignments; therefore, **no frontend implementation** is required for this submission.

1. Server Configuration (10 Marks):

- Create a **server.js** file in root folder of your backend app. As the entry point of the application, this file should initialize an Express server on port **3000**, either by importing your Express configuration from another module you created or by configuring Express directly inside of it.
- Add and configure properly the following middleware to your Express configuration: `http-errors`, `morgan`, and `CORS`.
- Create a global error handler and register it as the last middleware of the Express configuration, so that any error caught can be sent as a response.

2. Database Preparation (20 Marks)

- On MongoDB Atlas (mongodb.com), create a cluster to use in this portfolio project.
- Using the NPM module Mongoose in your Express app, create a database connection function with a database named *portfolio* in the connection string. Ensure your application is connected to the MongoDB Atlas database create above (a), by logging a message in the server console.
- Using Mongoose, create the following models with their respective properties.

I. references

name: string
testimonial: string
position: string
company: string

II. projects

title: string
completion: Date
description: string
image: string

III. services

title: string
description: string

IV. users

firstname: string
lastname: string
email: string
password: string
created: Date
updated: Date

3. Controllers Implementation (40 Marks):

- Create controllers (separate files) implementing all CRUD operations for references, projects, services, and users. Save them in the controllers folder.
- In all the controllers mentioned above, the functions for **add** and **getById** must return a response body containing an object with the attributes *success*, *message*, and *data*, where *data* includes the object itself and an *id* attribute (not *_id*). Example:

```
{
  "success": true,
  "message": "Service added successfully.",
  "data": {
    "title": "Web Application Development",
    "description": "Development of your web app using the MERN stack.",
    "id": "69891397597faee30388c455"
  }
}
```

- c. In all the controllers mentioned above, the functions designed for **update** and **delete** should return in the body of the response an object with attributes, *success*, and *message*. Example:

```
{
  "success": true,
  "message": "Service updated successfully."
}
```

- d. All functions designed for **getAll** should return in the body of the response an object with attributes *success*, *message* and *data*, which should be a list of objects from the database containing the respective id attribute (not *_id*). Example:

```
{
  "success": true,
  "message": "Services list retrieved successfully.",
  "data": [
    {
      "title": "Web Application Development",
      "description": "Development of your web app using the MERN stack.",
      "id": "698902aa987427ffd92afac7"
    },
    {
      "title": "Backend Application Development",
      "description": "Development of Backend applications compatible with various frontend technologies.",
      "id": "69891609fe6902a846c7e1cf"
    }
  ]
}
```

4. Routers Implementation (10 Marks):

- a. Create routers (separate files) and define the routes (endpoints) for handling all CRUD operations for references, services, project, and users, as per the specification below:

references

HTTP Methods	URLs of the endpoint	Actions
GET	api/references	get all references
GET	api/references/:id	get references by id
POST	api/references	add new contact

PUT	api/references/:id	update contact by id
DELETE	api/references/:id	remove contact by id

projects

HTTP Methods	URLs of the endpoint	Actions
GET	api/projects	get all projects
GET	api/projects/:id	get projects by id
POST	api/projects	add new project
PUT	api/projects/:id	update project by id
DELETE	api/projects/:id	remove project by id

services

HTTP Methods	URLs of the endpoint	Actions
GET	api/services	get all services
GET	api/services/:id	get services by id
POST	api/services	add new service
PUT	api/services/:id	update service by id
DELETE	api/services/:id	remove service by id

users

HTTP Methods	URLs of the endpoint	Actions
GET	api/users	get all users
GET	api/users/:id	get users by id
POST	api/users	add new user
PUT	api/users/:id	update user by id
DELETE	api/users/:id	remove user by id

5. Test the Backend Application (10 Marks):

- Download the file **Assignment 2.postman_collection.json**, import it into your Postman workspace, and run tests for all endpoints (three-dot menu → Run).
- Take screenshots of all test results, place them in a PDF, and attach it to your submission.

6. Share your files on **GitHub** to demonstrate Version Control Best Practices and **deploy** your site to a cloud host (20 Marks).

- Your repository must include your code and be well structured.
- Your repository must include commits that demonstrate the project being updated at different stages of development – each time a major change is implemented.
- You must deploy your site to your Cloud Server. Examples: Render, Vercel, Netlify or any other one.

SUBMITTING YOUR WORK

Your submission should include:

- A zip archive of your Backend project files. (Mandatory)
- A link to GitHub repository of your backend.
- A link to the live backend server running on a cloud provider (Ex.: Render.com)
- A PDF file from the results of the test.

This assignment is worth **5%** of your final course grade. Late submissions: **20%** deducted.